

Fine-tuning de wav2vec 2.0 con métodos PEFT para clasificación binaria de habla neurodegenerativa

Adaptadores bottleneck, LoRA y prefix-tuning — implementación y estado del arte

Informe técnico para tesis de maestría • Reconocimiento de patrones y de voz • Versión con glosario de siglas y citas referenciadas

Nota sobre este documento. Esta versión explica cada sigla la primera vez que aparece, incluye un glosario completo al inicio, y referencia cada afirmación a su fuente mediante el formato autor-año entre paréntesis. La bibliografía numerada completa está al final. Cuando la fuente es documentación técnica oficial (HuggingFace, AdapterHub) se cita por su nombre.

¿Qué es «TL;DR»? Es una sigla del inglés «*Too Long; Didn't Read*» (literalmente, «demasiado largo; no lo leí»). Se usa para encabezar un **resumen ejecutivo**: la síntesis de las conclusiones principales para quien no va a leer el documento completo. En este informe lo traducimos como **Resumen ejecutivo**.

Glosario de siglas y acrónimos

Ordenadas alfabéticamente. Cada una se explica también en su primer uso dentro del texto.

Sigla	Significado y aclaración
AD	<i>Alzheimer's Disease</i> — enfermedad de Alzheimer. La clase «positiva» en tu clasificación binaria.
ALS / ELA	<i>Amyotrophic Lateral Sclerosis</i> — Esclerosis Lateral Amiotrófica, enfermedad neurodegenerativa motora.
ASR	<i>Automatic Speech Recognition</i> — Reconocimiento Automático del Habla (transcribir audio a texto). No es tu tarea.
AUC	<i>Area Under the (ROC) Curve</i> — Área Bajo la Curva ROC; métrica de desempeño de clasificadores, de 0.5 (azar) a 1.0 (perfecto).
BA	<i>Bottleneck Adapter</i> — adaptador de cuello de botella (Método 1 de este informe).
BERT	<i>Bidirectional Encoder Representations from Transformers</i> — modelo de lenguaje preentrenado para texto.
CNN	<i>Convolutional Neural Network</i> — Red Neuronal Convolucional; el codificador de características de wav2vec.
CTC	<i>Connectionist Temporal Classification</i> — función de pérdida para alinear audio y texto en ASR. No aplica a clasificación.
CTR	<i>Control</i> — hablante sano (clase «negativa») en tu clasificación binaria.
CV	<i>Cross-Validation</i> — Validación Cruzada (p. ej. 5-fold = en 5 particiones).
D	Dimensión oculta del modelo (en wav2vec2-base, D = 768).
eGeMAPS	<i>extended Geneva Minimalistic Acoustic Parameter Set</i> — conjunto estandarizado de rasgos acústicos clásicos.
F1	Métrica que combina precisión y exhaustividad (media armónica). «F1-macro» la promedia por clase.
FFN	<i>Feed-Forward Network</i> — Red Prealimentada; el bloque denso dentro de cada capa Transformer.
fp16	<i>floating point 16-bit</i> — precisión de punto flotante de 16 bits (precisión mixta), ahorra memoria y acelera.
FT	<i>Fine-Tuning</i> — ajuste fino. «Full FT» = ajuste fino completo (todos los pesos).
GeLU	<i>Gaussian Error Linear Unit</i> — función de activación no lineal.
GPT	<i>Generative Pre-trained Transformer</i> — familia de modelos de lenguaje generativos.
GPU	<i>Graphics Processing Unit</i> — Unidad de Procesamiento Gráfico; el hardware de entrenamiento.
HC	<i>Healthy Control</i> — control sano (sinónimo de CTR en la literatura de Parkinson).
H&Y	<i>Hoehn & Yahr</i> — escala de severidad de la enfermedad de Parkinson.
Hz / kHz	Hertz / kilohertz — unidad de frecuencia. El audio se procesa a 16 kHz (16 000 muestras/segundo).
K / V	<i>Key / Value</i> — Clave / Valor; las dos proyecciones de la atención donde prefix-tuning inserta sus prefijos.
LoRA	<i>Low-Rank Adaptation</i> — Adaptación de Rango Bajo (Método 2 de este informe).
LR	Ambiguo según contexto: <i>Learning Rate</i> (tasa de aprendizaje) o <i>Logistic Regression</i> (regresión logística). Se aclara en cada uso.
lvPPA / PPA	<i>(logopenic variant) Primary Progressive Aphasia</i> — Afasia Progresiva Primaria (variante logopénica), trastorno neurodegenerativo del lenguaje.
m	Dimensión del cuello de botella en un adaptador (m < D).
MCI	<i>Mild Cognitive Impairment</i> — Deterioro Cognitivo Leve; estadio intermedio entre cognición normal y demencia.
MFCC	<i>Mel-Frequency Cepstral Coefficients</i> — rasgos acústicos clásicos previos al aprendizaje profundo.

MLP	Multi-Layer Perceptron — Perceptrón Multicapa; pequeña red densa.
MMS	Massively Multilingual Speech — modelo de habla multilingüe de Meta.
MMSE	Mini-Mental State Examination — test clínico de cribado cognitivo (0-30 puntos).
NC	Normal Cognition — Cognición Normal (control en estudios de MCI).
OOM	Out Of Memory — error de memoria insuficiente en la GPU.
PD	Parkinson's Disease — enfermedad de Parkinson.
PEFT	Parameter-Efficient Fine-Tuning — Ajuste Fino Eficiente en Parámetros; familia de métodos (adapters, LoRA, prefix-tuning) que entrenan solo una fracción de los pesos. También es el nombre de la librería de HuggingFace que los implementa.
r	Rango (en LoRA): dimensión de la descomposición de rango bajo.
RAB	Residual Adapter Block — Bloque Adaptador Residual.
ReLU	Rectified Linear Unit — función de activación no lineal (max(0,x)).
RF	Random Forest — Bosque Aleatorio; clasificador clásico de conjunto.
RMSE	Root Mean Square Error — Raíz del Error Cuadrático Medio; métrica de error en regresión.
ROC	Receiver Operating Characteristic — curva que relaciona sensibilidad y especificidad.
SER	Speech Emotion Recognition — Reconocimiento de Emociones en Habla; tarea de clasificación análoga a la tuya.
SPGC	Signal Processing Grand Challenge — Gran Reto de Procesamiento de Señales (de ICASSP).
SSL	Self-Supervised Learning — Aprendizaje Auto-Supervisado; el paradigma de preentrenamiento de wav2vec.
SVM	Support Vector Machine — Máquina de Vectores de Soporte; clasificador clásico.
TOPSIS	Technique for Order of Preference by Similarity to Ideal Solution — método de ranking multicriterio.
UPDRS	Unified Parkinson's Disease Rating Scale — escala unificada de evaluación de la enfermedad de Parkinson.
W2V / wav2vec	Nombre propio del modelo auto-supervisado de habla de Meta AI. «wav2vec 2.0» es la versión que usas.
WER	Word Error Rate — Tasa de Error de Palabra; métrica de ASR.
WS	Weighted Sum — Suma Ponderada (de las salidas de las capas Transformer).
XLSR	Cross-Lingual Speech Representations — variante multilingüe de wav2vec 2.0.

Conferencias y series citadas: ICASSP (International Conference on Acoustics, Speech and Signal Processing); Interspeech (congreso de tecnologías del habla); NeurIPS (Neural Information Processing Systems); ICML (International Conference on Machine Learning); ICLR (International Conference on Learning Representations); ACL/EACL/EMNLP (congresos de lingüística computacional); TSD (Text, Speech and Dialogue); LNCS (Lecture Notes in Computer Science, serie editorial de Springer).

Resumen ejecutivo (TL;DR)

- **Sí es viable y está bien fundamentado** hacer ajuste fino eficiente en parámetros (PEFT) de wav2vec 2.0 para clasificación binaria (p. ej. AD vs. CTR) con pocos datos. La clase correcta en la librería HuggingFace es `Wav2Vec2ForSequenceClassification` — una cabeza lineal sobre un *pooling*—, **no** `Wav2Vec2ForCTC`, que es para ASR ([documentación de HuggingFace Transformers \[13\]](#)). Varios de los tutoriales que aportaste enseñan CTC/ASR y no aplican directamente a tu tarea.
- Para los **tres métodos** que mapeaste: (1) los **adaptadores bottleneck** —arquitectura D→m→D con conexión residual e inicialización casi nula— se aplican con la librería `adapters` (Houlsby et al. 2019 [2]; Poth et al. 2023 [7]); (2) **LoRA** se aplica con la librería `peft` sobre `q_proj / v_proj`, reduciendo los parámetros entrenables en órdenes de magnitud (Hu et al. 2021 [3]); (3) **prefix-tuning** antepone vectores continuos entrenables en cada capa (Li & Liang 2021 [4]). La evidencia empírica en habla muestra que con **2-3 % de los parámetros** se iguala o supera el ajuste fino completo en clasificación (Feng et al. 2024 [9]; Inoue et al. 2024 [10]).
- **Estado del arte en enfermedades neurodegenerativas:** existe trabajo abundante con wav2vec 2.0 / HuBERT / WavLM en Alzheimer (corpus ADReSS/ADReSSo/Pitt), Parkinson (corpus PC-GITA) y deterioro cognitivo leve (MCI), casi siempre con **embeddings congelados o ajuste fino completo**. El uso de **PEFT específicamente para clasificación neurodegenerativa es una laguna casi inexplorada** —solo dos preprints de 2026, HASS y VoxCog (Li et al. 2026 [18]; Feng et al. 2026 [19])—, lo que convierte tu tesis en una contribución oportuna y original.

Hallazgos clave

1. **Clasificación ≠ ASR.** Tu tarea es *secuencia → etiqueta única* (binaria). Se usa `Wav2Vec2ForSequenceClassification`, que aplica una proyección, hace *mean pooling* (promedio sobre el eje temporal) y pasa por una capa densa más una capa de clasificación ([documentación de HuggingFace Transformers \[13\]](#); [notebook oficial de audio classification de HuggingFace \[14\]](#)). Esto difiere del pipeline

CTC de tus tutoriales de referencia.

- 2. **El codificador CNN debe congelarse** (`model.freeze_feature_encoder()`). Es la práctica estándar; Vaessen & van Leeuwen (2022) mostraron que congelarlo da resultados más estables entre semillas aleatorias (Vaessen & van Leeuwen 2022 [11]).
- 3. **wav2vec2-base**: CNN de 7 capas (16 kHz → ~50 Hz, un vector cada 20 ms), **12 capas Transformer**, dimensión oculta **768**, 12 cabezas de atención, FFN intermedia de 3072, **~95 millones de parámetros**. La variante *large* tiene 24 capas y dimensión 1024 (~315 M) (Baevski et al. 2020 [1]; Cohen 2024 [12]).
- 4. **Las capas tempranas importan en habla clínica**. Varios estudios encuentran que las **primeras capas Transformer**, o incluso el codificador de características, capturan la información patológica más discriminativa (Klempíř et al. 2025 [16]; estudios de disartria, ar5iv 2309.14107 [27]).
- 5. **PEFT iguala o supera al ajuste fino completo en clasificación de habla con una fracción de los parámetros**. En reconocimiento de emociones (SER), la combinación de adaptador bottleneck + LoRA + suma ponderada supera al ajuste fino completo usando **2-3 % de los parámetros entrenables** (Feng et al. 2024, PEFT-SER [9]).
- 6. **Riesgo metodológico crítico — efecto «Clever Hans» y fuga de datos**. El ajuste fino de wav2vec 2.0 para AD puede alcanzar 97.2 % en el corpus Pitt pero desplomarse a 77.7 % en ADReSSo, evidenciando sobreajuste a artefactos del corpus y no a biomarcadores reales (Liu et al. 2024 [15]).

PARTE 1 — Implementación y estado del arte del fine-tuning con PEFT

1.1 La distinción fundamental: clasificación vs. transcripción

Conviene aclararlo de entrada porque condiciona todo el código. ASR (*Automatic Speech Recognition*, reconocimiento automático del habla) transcribe audio a texto; tu tarea, en cambio, asigna **una etiqueta** a toda la grabación. Los recursos que aportaste se dividen así:

- **heyamit10 (Medium), khanld/ASR-Wav2vec-Finetune, TensorFlow Hub** → enseñan **ASR** con pérdida CTC (*Connectionist Temporal Classification*), usando la clase `Wav2Vec2ForCTC`. El objetivo es alinear audio con texto. **No es tu caso**, aunque son útiles para entender el preprocesamiento de audio (carga, resampleo a 16 kHz, conversión a mono).
- **Tu tarea (AD vs. CTR)** → clasificación de secuencia. La clase es `Wav2Vec2ForSequenceClassification`, descrita en la documentación oficial como un modelo wav2vec 2.0 con una cabeza de clasificación (capa lineal sobre la salida agrupada) para tareas como el reconocimiento de palabras clave de SUPERB (documentación de HuggingFace Transformers [13]).

El notebook oficial de HuggingFace `examples/audio_classification.ipynb` es el ejemplo canónico para tu pipeline: sirve para cualquier checkpoint de habla que tenga una versión con cabeza de clasificación de secuencia, como `Wav2Vec2ForSequenceClassification` (notebook de HuggingFace [14]).

```
Audio crudo 16 kHz
→ CNN feature encoder (7 capas conv, CONGELADO)
→ Proyección 512 → 768
→ 12 capas Transformer (dim 768) ← aquí se inserta el PEFT
→ Mean pooling temporal
→ Capa densa de proyección
→ Cabeza lineal (num_labels = 2)
→ Logits: AD vs. CTR
```

1.2 Arquitectura de wav2vec 2.0 base

Lo que necesitas saber para insertar PEFT. Todas las cifras provienen del artículo original (Baevski et al. 2020 [1]) y de descripciones técnicas de la arquitectura (Cohen 2024 [12]).

Componente	Detalle	¿Se entrena?
Feature encoder (CNN)	7 capas convolucionales; 16 kHz → ~50 Hz (1 vector / 20 ms); salida 512-dim	Congelado (estándar)
Proyección	512 → 768 (primer estado oculto, índice 0)	Según estrategia
Red de contexto	12 capas Transformer , dim 768 , 12 cabezas, FFN 3072	Aquí van adapters / LoRA / prefijos
Pooling	Mean pooling sobre el eje temporal	—
Cabeza de clasificación	Densa (p. ej. 256) + capa lineal a <code>num_labels</code>	Siempre se entrena

Cada capa Transformer contiene proyecciones de atención llamadas `q_proj` (query/consulta), `k_proj` (key/clave), `v_proj` (value/valor) y `out_proj` (salida), más el bloque prealimentado (`intermediate_dense`, `output_dense`). Estos son los nombres de módulo que necesitas para apuntar LoRA o adaptadores.

1.3 Pipeline base (ajuste fino completo o parcial)

```
import torch
from transformers import (Wav2Vec2ForSequenceClassification,
                          AutoFeatureExtractor, TrainingArguments, Trainer)

model_name = "facebook/wav2vec2-base"
feature_extractor = AutoFeatureExtractor.from_pretrained(model_name)

model = Wav2Vec2ForSequenceClassification.from_pretrained(
    model_name,
    num_labels=2,                # AD vs CTR
    label2id={"CTR": 0, "AD": 1},
    id2label={0: "CTR", 1: "AD"},
)

# Practica estandar: congelar el feature encoder CNN
model.freeze_feature_encoder()

# Preprocesamiento (audio ya cargado como np.array mono a 16 kHz)
def preprocess(batch):
    inputs = feature_extractor(
        batch["audio"], sampling_rate=16000,
        max_length=16000*10, truncation=True, padding="max_length")
    inputs["labels"] = batch["label"]
    return inputs
```

Ajuste fino parcial (recomendado con pocos datos): en vez de entrenar las 12 capas, descongela solo las **últimas 3 capas Transformer**. Sampath et al. (2025) —artículo que ya tienes en tu bibliografía— demuestran que el ajuste fino parcial alcanza un desempeño comparable al completo sin diferencias estadísticamente significativas, recomendando entrenar las últimas tres capas en precisión mixta (fp16), con 67 % de aceleración, y añadir *caching* de representaciones intermedias (88 % de aceleración, −71 % de parámetros entrenables) (Sampath et al. 2025 [8]).

```
# Fine-tuning parcial: congelar todo salvo las ultimas 3 capas + cabeza
for p in model.wav2vec2.parameters():
    p.requires_grad = False
for layer in model.wav2vec2.encoder.layers[-3:]:
    for p in layer.parameters():
        p.requires_grad = True
# La cabeza (model.classifier / model.projector) queda entrenable por defecto
```

1.4 Método 1 — Adaptadores bottleneck (Houlsby et al. 2019)

Mecanismo. Un adaptador es un módulo de cuello de botella (*bottleneck*) insertado dentro de cada capa Transformer. Dada una representación oculta h de dimensión D :

$$h \leftarrow W_{up} \cdot f(W_{down} \cdot h) + r$$

donde W_{down} (de tamaño $m \times D$) proyecta a una dimensión reducida $m \ll D$ (el cuello de botella), f es una no-linealidad (ReLU o GeLU), W_{up} (de tamaño $D \times m$) re-proyecta a D , y r es la **conexión residual**. La **inicialización casi a cero** de W_{up} hace que al inicio el adaptador sea aproximadamente la identidad, preservando el modelo preentrenado; la documentación de AdapterHub confirma que el módulo insertado debe ser equivalente a la función identidad al comenzar el ajuste fino, condición clave para un buen desempeño (Houlsby et al. 2019 [2]; documentación de AdapterHub [21]).

¿Dónde se inserta?

- **Configuración Houlsby** (`DoubleSeqBnConfig`): **dos** adaptadores por capa —tras la atención multi-cabeza y tras el bloque prealimentado—. Más expresivo, $\sim 2 \times$ parámetros (Houlsby et al. 2019 [2]).
- **Configuración Pfeiffer** (`SeqBnConfig`): **un** adaptador por capa, solo tras el FFN. Más eficiente; es el default en la mayoría de frameworks y el usado en los benchmarks de audio (documentación de AdapterHub [21]; Pfeiffer et al., AdapterFusion [6]).

Presupuesto de parámetros. Controlado por el factor de reducción (*reduction factor* = D/m). Con $D = 768$ y factor 16, $m = 48$; cada adaptador añade aproximadamente $2 \cdot D \cdot m \approx 74\,000$ parámetros, ~ 0.9 M en 12 capas (Pfeiffer) —del orden del **~ 1 %** del modelo—. Houlsby duplica esa cifra. En la adaptación de Whisper-small a habla infantil, el adapter tuning logró un desempeño similar al ajuste fino completo actualizando solo 1.29 M de parámetros (benchmark de ASR infantil, arXiv 2406.10507 [22]).

```
# pip install adapters
import adapters
from adapters import BnConfig
from transformers import Wav2Vec2ForSequenceClassification

model = Wav2Vec2ForSequenceClassification.from_pretrained(
    "facebook/wav2vec2-base", num_labels=2)
```

```

adapters.init(model)                # habilita la API de adapters

# Bottleneck Houlby: adapter tras atencion Y tras FFN
config = BnConfig(
    mh_adapter=True,                # adapter tras multi-head attention
    output_adapter=True,            # adapter tras FFN -> configuracion Houlby
    reduction_factor=16,            # D/m (768/16 = 48 dim de cuello de botella)
    non_linearity="relu",
)
model.add_adapter("ad_vs_ctr", config=config)
model.train_adapter("ad_vs_ctr")     # congela el resto; entrena adapter + cabeza

```

Para la variante Pfeiffer, usa `mh_adapter=False, output_adapter=True`. La librería `adapters` soporta wav2vec 2.0 oficialmente (Poth et al. 2023 [7]).

1.5 Método 2 — LoRA, Adaptación de Rango Bajo (Hu et al. 2021)

Mecanismo. En lugar de actualizar una matriz de pesos W_0 (de tamaño $d \times k$), LoRA la congela y aprende una actualización de **rango bajo**:

$$W = W_0 + \Delta W = W_0 + B \cdot A, \quad \text{con } B (d \times r), A (r \times k), r \ll \min(d, k)$$

El paso hacia adelante es $h = W_0 x + B A x$, escalado por α/r . La hipótesis fundacional es que la actualización de pesos durante la adaptación tiene un «**rango intrínseco**» **bajo**. A se inicializa con ruido gaussiano y B a cero, de modo que $\Delta W = 0$ al inicio. Ventaja única frente a adaptadores y prefijos: en inferencia, BA puede **fusionarse** en W_0 sin sobre coste de latencia (Hu et al. 2021 [3]).

¿Dónde se inserta?

Lo estándar es aplicarlo a las proyecciones de atención `q_proj` y `v_proj`. En habla, el trabajo de detección de audio falso con wav2vec2/XLSR encontró que **rango $r = 4$** y aplicar LoRA a las matrices de consulta y valor (o consulta, clave y valor) era óptimo, reduciendo los parámetros entrenables $\sim 198 \times$ (-99.49%) con una caída de desempeño marginal frente al ajuste fino completo (Wang et al. 2023 [20]).

Presupuesto de parámetros. Por cada capa adaptada: $r(d+k)$ parámetros. Con $d = k = 768$ y $r = 16$: $16 \cdot (768 + 768) \approx 24\,600$ por proyección; aplicado a `q_proj + v_proj` en 12 capas $\approx 590\,000$, **<1 %** del modelo. Reducir a $r = 8$ o $r = 4$ lo baja a la mitad o a la cuarta parte (Hu et al. 2021 [3]; Wang et al. 2023 [20]).

```

# pip install peft
from peft import LoraConfig, get_peft_model
from transformers import Wav2Vec2ForSequenceClassification

model = Wav2Vec2ForSequenceClassification.from_pretrained(
    "facebook/wav2vec2-base", num_labels=2)
model.freeze_feature_encoder()

lora_config = LoraConfig(
    r=16,                                # rango (prueba 4, 8, 16)
    lora_alpha=32,                        # escalado alpha/r
    target_modules=["q_proj", "v_proj"],  # proyecciones de atencion
    lora_dropout=0.05,
    bias="none",
    modules_to_save=["classifier", "projector"], # la cabeza se entrena completa
)
model = get_peft_model(model, lora_config)
model.print_trainable_parameters()
# -> trainable params: ~0.6M || all params: ~95M || trainable%: ~0.6%

```

Nota práctica: como wav2vec2 no es un modelo de lenguaje causal, **no** fijas `task_type="CAUSAL_LM"`; déjalo sin especificar y usa `modules_to_save` para que la cabeza de clasificación (inicializada aleatoriamente) se entrene por completo. Este patrón —LoRA sobre `q_proj / v_proj` de un modelo wav2vec2 de HuggingFace— está documentado en el propio repositorio de `peft` (HuggingFace PEFT, issue #1569 [23]).

1.6 Método 3 — Prefix-tuning (Li & Liang 2021)

Mecanismo. Antepone una secuencia de **vectores continuos entrenables** («prefijos» o *soft prompts*) a los estados ocultos de **cada** capa Transformer, manteniendo congelados todos los pesos del modelo. A diferencia del *prompt-tuning*, que solo añade prompts en la capa de entrada, prefix-tuning optimiza pares clave-valor de prefijo en **todas** las capas. Para estabilizar el entrenamiento, los prefijos no se optimizan directamente sino que se reparametrizan vía un MLP (que se descarta tras entrenar). Es muy eficiente en almacenamiento: solo guardas los vectores de prefijo (Li & Liang 2021 [4]).

¿Dónde se inserta?

En las claves (K) y valores (V) de la atención de cada capa: se concatenan L_{prefix} vectores antes de las claves y valores reales. En audio se ha usado prefix-tuning sobre wav2vec2 para inyectar información de palabra, con tan solo $L_{\text{prefix}} = 2$ vectores de prefijo (NOCASA 2025, arXiv 2509.03256 [25]).

Presupuesto de parámetros. $L_{\text{prefix}} \times (\text{n.º de capas}) \times 2 \times D$ (el factor 2 por clave y valor). Con $L_{\text{prefix}} = 16$, 12 capas y $D = 768$: $\approx 295\,000$ parámetros ($\sim 0.3\%$), más el MLP temporal de reparametrización (Li & Liang 2021 [4]).

```
from peft import PrefixTuningConfig, get_peft_model

prefix_config = PrefixTuningConfig(
    num_virtual_tokens=16,      # L_prefix: vectores de prefijo por capa
    encoder_hidden_size=768,
)
model = get_peft_model(model, prefix_config)
model.print_trainable_parameters()
```

Advertencia honesta sobre prefix-tuning. Su soporte en la librería `peft` está orientado a modelos de lenguaje; para wav2vec2 puede requerir ajustes, o usar la librería `adapters` (que implementa Prefix Tuning de Li & Liang explícitamente [7]). Si encuentras fricción, prioriza LoRA y los adaptadores bottleneck, que tienen soporte de primera clase para wav2vec2. Además, la búsqueda bibliográfica **no halló ningún trabajo** que use prefix-tuning para clasificación neurodegenerativa, así que es la opción más experimental de las tres.

1.7 Tabla comparativa de los tres métodos PEFT

Método	Mecanismo	Inserción en wav2vec2	% parám. típico	Librería	¿Fusionable en inferencia?
Adaptador bottleneck	$h \leftarrow W_{\text{up}} \cdot f(W_{\text{down}} \cdot h) + r$	Tras atención y/o FFN	$\sim 1\text{--}2\%$	<code>adapters</code>	No
LoRA	$W_0 + (\alpha/r) \cdot BA$	<code>q_proj</code> , <code>v_proj</code>	$< 1\%$	<code>peft</code>	Sí
Prefix-tuning	prefijos K/V por capa	claves/valores de atención	$\sim 0.3\%$	<code>peft</code> / <code>adapters</code>	No

Fuentes de la tabla: Houlsby et al. 2019 [2]; Hu et al. 2021 [3]; Li & Liang 2021 [4]; Poth et al. 2023 [7].

1.8 Bucle de entrenamiento común

```
import numpy as np, evaluate
from transformers import TrainingArguments, Trainer

acc = evaluate.load("accuracy"); f1 = evaluate.load("f1")
def compute_metrics(p):
    preds = np.argmax(p.predictions, axis=1)
    return {"accuracy": acc.compute(predictions=preds, references=p.label_ids)["accuracy"],
            "f1": f1.compute(predictions=preds, references=p.label_ids, average="macro")["f1"]}

args = TrainingArguments(
    output_dir="w2v2-ad-ctr", learning_rate=3e-4,  # PEFT tolera LR mayor que full FT (3e-5)
    per_device_train_batch_size=4, gradient_accumulation_steps=4,
    num_train_epochs=15, warmup_ratio=0.1, eval_strategy="epoch",
    fp16=True, load_best_model_at_end=True, metric_for_best_model="f1")

trainer = Trainer(model=model, args=args, compute_metrics=compute_metrics,
                  train_dataset=train_ds, eval_dataset=eval_ds)
trainer.train()
```

Notas: para ajuste fino completo usa una tasa de aprendizaje (LR) $\approx 3 \times 10^{-5}$; para LoRA y adaptadores puedes subir a 1×10^{-4} – 5×10^{-4} . En datasets pequeños, **batch de tamaño 1 + acumulación de gradiente de 4 y validación cruzada 5-fold por hablante** es el protocolo usado en la literatura de AD (Liu et al. 2024 [15]).

PARTE 2 — Estado del arte: modelos auto-supervisados de habla en enfermedades neurodegenerativas

2.1 Panorama general y datasets clave

Dataset	Enfermedad	Idioma	Contenido	Acceso
ADReSS (Interspeech 2020)	AD	Inglés	Cookie Theft, balanceado edad/género	TalkBank/DementiaBank

ADReSSo (Interspeech 2021)	AD	Inglés	Solo audio (sin transcripción manual)	DementiaBank
ADReSS-M (ICASSP 2023)	AD	Inglés (train) / Griego (test)	Multilingüe, 291 muestras	DementiaBank
DementiaBank / Pitt	AD / demencia	Inglés	Habla espontánea longitudinal	DementiaBank
PC-GITA	Parkinson	Español (Colombia)	50 PD + 50 HC; vocales/lectura/monólogo	Bajo solicitud (Orozco-Arroyave, U. de Antioquia)
PROCESS (ICASSP 2025)	MCI + demencia	Inglés	82 HC, 59 MCI, 16 demencia; 3 prompts	Challenge
TAUKADIAL (Interspeech 2024)	MCI / NC	Inglés + Mandarín	507 muestras (285 MCI, 222 NC)	Challenge

PC-GITA es especialmente relevante por su origen colombiano (GITA Lab, Universidad de Antioquia, Medellín): 50 pacientes con PD (UPDRS total medio 37.7 ± 18.3 ; H&Y 2.2 ± 0.7) y 50 controles, balanceado por sexo y edad (61 ± 9 años), con tareas de lectura, monólogo espontáneo y vocales sostenidas (Klempíř et al. 2025 [16]; Gallo-Aristizábal et al. 2024 [17]).

2.2 Alzheimer y demencia (AD)

- **Agbavor & Liang (2022), PLOS Digital Health.** Usaron wav2vec2-base-960h vía Wav2Vec2ForCTC para transcribir el audio, y luego embeddings de GPT-3 + SVM/regresión logística/Random Forest para clasificar. Es un esquema de *embedding como rasgo*, no ajuste fino end-to-end del modelo de habla. El propio artículo señala que estos estudios de detección de Alzheimer basados en habla con IA han alcanzado exactitudes cercanas al 90 % en promedio (Agbavor & Liang 2022 [24]).
- **Zhu, Obyat, Liang, Batsis & Roth (2021), Interspeech — WavBERT.** Convierten wav2vec → transcripción → BERT, con una red de conversión que preserva información no-semántica (pausas). Reportan una exactitud máxima del 83.1 % en clasificación sobre ADReSSo (Zhu et al. 2021 [25]).
- **Liu, Feng, Yuan & Ling (2024) — efecto Clever Hans.** Ajuste fino end-to-end de wav2vec2-large-xlsr-53 (inglés) y wav2vec2-large-chinese (mandarín) con cabeza de clasificación de secuencia (capa lineal + sigmoide sobre average pooling), 5-fold CV, batch 1, acumulación 4, 15 épocas, $LR \times 10^{-5}$. Hallazgo crítico de fuga de datos: la exactitud intra-corpus en Pitt se acerca al 97.2 %, muy por encima del 80.7 % y 77.7 % en los datasets del reto (ADReSS/ADReSSo). **Lección directa para tu tesis:** la altísima exactitud intra-corpus puede reflejar artefactos del corpus (Clever Hans), no biomarcadores reales; valida siempre cross-corpus y por hablante (Liu et al. 2024 [15]).

2.3 Parkinson (PD)

- **Gallo-Aristizábal et al. (2024), TSD — GITA Lab.** Con wav2vec 2.0 sobre PC-GITA reportan exactitudes de 86 %, 80 % y 83 % a nivel de fonema, sílaba y palabra, respectivamente (Gallo-Aristizábal et al. 2024 [17]). (Cifras más altas que circulan, como «92 % en vocales / 97 % en palabras», corresponden a otros métodos multimodales del mismo grupo, no a wav2vec puro; no las atribuyas a wav2vec.)
- **Klempíř et al. (2025), medRxiv.** Comparan wav2vec 1.0 vs. 2.0 (embeddings congelados + clasificadores clásicos, ranking TOPSIS) en 3 corpus multilingües incluido PC-GITA. wav2vec 2.0 destaca en todos los modos: su **primera capa Transformer** rinde mejor en lectura y monólogo, y su **feature extractor** rinde mejor en clasificación por vocales. **Implicación de diseño:** la primera capa Transformer y el codificador de características son las representaciones más informativas para PD (Klempíř et al. 2025 [16]).
- **Pooling de embeddings (medRxiv 2025).** Las capas tempranas de wav2vec2 dan los rasgos más informativos; el **mean pooling** resultó una de las estrategias de agregación más efectivas y estables; y el modelo afinado para disartria no superó consistentemente al base. Útil para justificar tu elección de pooling (estudio de layer pooling para PD, ScienceDirect S0010482525017226 [26]).

2.4 Deterioro cognitivo leve (MCI) y demencia multiclase

- **PROCESS Grand Challenge (ICASSP 2025) —** primer reto que añade MCI como clase (clasificación de 3 clases: sano / MCI / demencia, más regresión de MMSE). El equipo ganador integró rasgos interpretables y temporales de modelos preentrenados, logrando $F1 = 0.649$ en clasificación y $RMSE = 2.628$ en MMSE; otros equipos combinaron eGeMAPS + HuBERT + GPT-4o superando los baselines de WavLM, HuBERT y Wav2Vec 2.0 (reto PROCESS / Tao et al. 2025 [28]; arXiv 2412.09928 [29]).
- **Zafar et al. (ICASSP 2025).** Ensemble que alcanza F1-macro de 0.96 en validación y 0.99 en 5-fold CV, pero **solo 0.64 en test** —otra ilustración del salto validación→test en datos clínicos pequeños— (Zafar et al. 2025 [30]).

2.5 Disartria y habla patológica (puente hacia neurodegenerativas motoras)

La disartria es transversal a PD, ALS y parálisis cerebral, y es el dominio donde más se ha explorado PEFT sobre SSL:

- Detección/severidad de disartria con wav2vec2: mejores resultados con embeddings de la **primera capa** (ar5iv 2309.14107 [27]).
- Adaptación estructurada con **bloques adaptadores residuales bottleneck (RAB)** sobre HuBERT-large y wav2vec2-conformer, con dimensión de cuello de botella 128-256. Es PEFT sobre SSL en datos clínicos, aunque para ASR, no clasificación de enfermedad (arXiv 2412.18832 [31]).

2.6 PEFT para clasificación neurodegenerativa: la laguna que tu tesis puede llenar

Este es el hallazgo más importante para tu posicionamiento. Tras una búsqueda dedicada, **la intersección {PEFT} × {SSL de habla} × {clasificación de enfermedad neurodegenerativa} es casi un vacío:**

- **HASS** (Li et al. 2026, preprint) — detección de **afasia progresiva primaria logopénica (lvPPA)**, trastorno neurodegenerativo. Aplica **LoRA sobre wav2vec 2.0 base, exclusivamente en las proyecciones de consulta y valor** de la auto-atención. AUC 0.892 ± 0.076 , F1 0.800 ± 0.072 . **No reporta rango LoRA, ni % de parámetros, ni comparación PEFT vs. ajuste fino completo** (su contribución es el dato sintético). Es el match más limpio con tus criterios de modelo (Li et al. 2026 [18]).
- **VoxCog** (Feng et al. 2026, preprint) — clasificación de **AD y MCI** multilingüe con **LoRA + capas convolucionales 1D** sobre modelos de fundación. Caveat: usa **Whisper-Large y MMS, no wav2vec2/HuBERT/WavLM**. Exactitud 87.5 % / F1 0.875 en ADReSS 2020; 85.9 % / 0.859 en ADReSSo. Tampoco reporta rango ni comparación PEFT vs. completo (Feng et al. 2026 [19]).

Plantilla metodológica de referencia. Como ninguno de los dos detalla rango/parámetros, el precedente cuantitativo más sólido de «LoRA sobre wav2vec2/XLSR» proviene de detección de audio falso: rango $r = 4$ óptimo, LoRA en consulta/valor, reducción $\sim 198\times$ de parámetros entrenables con caída marginal (Wang et al. 2023 [20]). Y los benchmarks de PEFT en habla (PEFT-SER; ELP-Adapters, que con WavLM iguala o supera al ajuste fino completo con 90 % menos parámetros) te dan el respaldo empírico de que la idea funciona en clasificación de habla (Feng et al. 2024 [9]; Inoue et al. 2024 [10]).

Recomendaciones

Etapla 0 — Reproduce el pipeline base (1-2 semanas). Monta `Wav2Vec2ForSequenceClassification` con `facebook/wav2vec2-base`, `freeze_feature_encoder()`, mean pooling y cabeza binaria. Verifica el flujo con un subconjunto pequeño (ADReSS o PC-GITA, este último vía Orozco-Arroyave en la U. de Antioquia, que te queda cerca). Si la GPU es limitada, empieza con **embeddings congelados + SVM/regresión logística** —rápido, pocos datos, baseline robusto que la literatura de PD usa con éxito— (Klempíř et al. 2025 [16]).

Etapla 1 — Establece baselines de ajuste fino. Compara, con la misma partición por hablante y 5-fold CV: (a) embeddings congelados + clasificador clásico; (b) ajuste fino parcial (últimas 3 capas + cabeza, fp16); (c) ajuste fino completo. Reporta exactitud, F1-macro y AUC. Si (b) iguala a (c) sin diferencia significativa, descarta el completo por coste (Sampath et al. 2025 [8]).

Etapla 2 — Introduce los tres métodos PEFT como tu contribución central. Implementa y compara LoRA ($r \in \{4, 8, 16\}$ sobre `q_proj/v_proj`), adaptadores bottleneck (Pfeiffer y Houlshby, factor 16) y prefix-tuning (8 y 16 tokens virtuales). Mide para cada uno F1/AUC frente al **% de parámetros entrenables** (esta curva eficiencia-desempeño es la figura estrella de tu tesis). El objetivo realista es igualar al ajuste fino completo usando **<3 %** de parámetros (Feng et al. 2024 [9]).

Etapla 3 — Valida contra el efecto Clever Hans. Haz al menos una evaluación **cross-corpus** o cross-lingüística. Si tu exactitud intra-corpus es $\sim 97\%$ pero cae a $\sim 60-77\%$ cross-corpus, repórtalo: es un resultado honesto y valioso, no un fracaso (Liu et al. 2024 [15]).

Etapla 4 — Posiciona tu originalidad. Dado que PEFT para clasificación neurodegenerativa con wav2vec2 está casi inexplorado (solo HASS y VoxCog en 2026, ninguno con comparación sistemática PEFT vs. completo ni reporte de rango/parámetros), una comparación rigurosa y reproducible de adapters vs. LoRA vs. prefix-tuning en régimen de pocos datos clínicos es una contribución publicable (apunta a Interspeech o al SPGC de ICASSP) (Li et al. 2026 [18]; Feng et al. 2026 [19]).

Decisiones por defecto recomendadas: modelo `wav2vec2-base` (no large, por datos limitados y hardware); congelar CNN; mean pooling (estable y simple, respaldado por la literatura de PD); LR 1×10^{-4} para PEFT; batch 1 + acumulación 4; 15 épocas con warmup 0.1; F1-macro como métrica de selección. Considera extraer la **primera capa Transformer** como representación alternativa, dado que varios estudios la señalan como la más informativa en habla patológica (Klempíř et al. 2025 [16]; ar5iv 2309.14107 [27]).

Advertencias (caveats)

- **Hardware y modelo:** el ajuste fino completo de wav2vec2-base puede dar error de memoria (OOM) en GPUs de consumo con batch ≥ 32 ; por eso PEFT y el ajuste fino parcial con fp16/caching son tu mejor opción en un PC personal (Sampath et al.

2025 [8]).

- **Soporte de prefix-tuning para wav2vec2 es el más frágil** de los tres en `peft` (orientado a modelos de lenguaje); si encuentras errores, usa la librería `adapters` o prioriza LoRA y adaptadores bottleneck. No se halló ningún trabajo previo de prefix-tuning para clasificación neurodegenerativa (Poth et al. 2023 [7]).
- **Riesgo de sobreajuste y fuga de datos:** con corpus de decenas de hablantes, la métrica intra-corpus engaña. Particiona **siempre por hablante** (nunca por grabación) para evitar que el modelo memorice voces. El salto validación→test (0.96→0.64) es la norma, no la excepción (Liu et al. 2024 [15]; Zafar et al. 2025 [30]).
- **Las cifras de la literatura no son directamente comparables:** difieren en corpus, idioma, tarea (binaria vs. 3 clases) y en si usan embeddings congelados, ajuste fino completo o rasgos lingüísticos. No mezcles un 97 % del Pitt corpus con un 64 % del PROCESS test como si midieran lo mismo (Liu et al. 2024 [15]; reto PROCESS [28]).
- **Los dos preprints PEFT-neurodegenerativos (HASS, VoxCog) son de 2026 y no estaban revisados por pares** al momento de esta búsqueda; cítalos como preprints de arXiv y verifica si hay versión publicada antes de la defensa (Li et al. 2026 [18]; Feng et al. 2026 [19]).
- **Acceso a datos:** ADReSS/ADReSSo/Pitt requieren membresía DementiaBank/TalkBank; PC-GITA es bajo solicitud al Prof. Juan Rafael Orozco-Arroyave (U. de Antioquia). Gestiona estos accesos con antelación (Gallo-Aristizábal et al. 2024 [17]).

Bibliografía

- [1] Baevski, A., Zhou, Y., Mohamed, A., & Auli, M. (2020). *wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations*. NeurIPS 33, 12449–12460. arXiv:2006.11477.
- [2] Houlisby, N., Giurugi, A., Jastrzebski, S., Morrone, B., de Laroussilhe, Q., Gesmundo, A., Attariyan, M., & Gelly, S. (2019). *Parameter-Efficient Transfer Learning for NLP*. ICML.
- [3] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv:2106.09685 (ICLR 2022).
- [4] Li, X. L., & Liang, P. (2021). *Prefix-Tuning: Optimizing Continuous Prompts for Generation*. ACL. arXiv:2101.00190.
- [5] Lester, B., Al-Rfou, R., & Constant, N. (2021). *The Power of Scale for Parameter-Efficient Prompt Tuning*. EMNLP.
- [6] Pfeiffer, J., et al. (2020/2021). *AdapterHub / AdapterFusion*. EMNLP / EACL.
- [7] Poth, C., Sterz, H., Paul, I., et al. (2023). *Adapters: A Unified Library for Parameter-Efficient and Modular Transfer Learning*. EMNLP (system demo). arXiv:2311.11077.
- [8] Sampath, A., Tavernor, J., & Mower Provost, E. (2025). *Efficient Finetuning for Dimensional Speech Emotion Recognition in the Age of Transformers*. arXiv:2503.03756.
- [9] Feng, T., et al. (2024). *PEFT-SER: On the Use of Parameter Efficient Transfer Learning Approaches for Speech Emotion Recognition Using Pre-trained Speech Models*. arXiv:2306.05350 (ACII).
- [10] Inoue, N., Otake, S., Hirose, T., Ohi, M., & Kawakami, R. (2024). *ELP-Adapters: Parameter Efficient Adapter Tuning for Various Speech Processing Tasks*. arXiv:2407.21066.
- [11] Vaessen, N., & van Leeuwen, D. A. (2022). *Fine-Tuning Wav2Vec2 for Speaker Recognition*. ICASSP. arXiv:2109.15053.
- [12] Cohen, M. X. (2024). *Exploring the Wav2vec2 Model*. (Análisis técnico de la arquitectura.)
- [13] HuggingFace. *Wav2Vec2 — Transformers documentation* (clase `Wav2Vec2ForSequenceClassification`). huggingface.co/docs/transformers.
- [14] HuggingFace. *Audio classification notebook* (`examples/audio_classification.ipynb`). github.com/huggingface/notebooks.
- [15] Liu, Y., Feng, et al. (2024). *Clever Hans Effect Found in Automatic Detection of Alzheimer's Disease through Speech*. arXiv:2406.07410.
- [16] Klempíř, O., Skryjová, A., Tichopad, A., & Krupička, R. (2025). *Ranking Pretrained Speech Embeddings in Parkinson's Disease Detection: Does Wav2Vec 2.0 Outperform its 1.0 Version Across Speech Modes and Languages?* medRxiv 2025.01.29.25321319.
- [17] Gallo-Aristizábal, J. D., Escobar-Grisales, D., Ríos-Urrego, C. D., Nöth, E., & Orozco-Arroyave, J. R. (2024). *Automatic Classification of Parkinson's Disease Using Wav2vec Embeddings at Phoneme, Syllable, and Word Levels*. TSD 2024, LNCS 15049. doi:10.1007/978-3-031-70566-3_27.
- [18] Li, H., et al. (2026). *HASS: Hierarchical Simulation of Logopenic Aphasic Speech for Scalable PPA Detection*. arXiv:2603.26795 (preprint).
- [19] Feng, T., Xu, A., Lee, J., & Narayanan, S. (2026). *VoxCog: Towards End-to-End Multilingual Cognitive Impairment Classification through Dialectal Knowledge*. arXiv:2601.07999 (preprint).
- [20] Wang, C., Yi, J., Zhang, X., Tao, J., Xu, L., & Fu, R. (2023). *Low-rank Adaptation Method for Wav2vec2-based Fake Audio Detection*. IJCAI 2023 Workshop (DADA). arXiv:2306.05617.
- [21] AdapterHub. *Adapter Methods documentation*. docs.adapterhub.ml/methods.html.
- [22] (2024). *Benchmarking Children's ASR with Supervised and Self-supervised Speech Foundation Models*. arXiv:2406.10507.
- [23] HuggingFace PEFT. *Issue #1569: LoRA with Wav2Vec2 base model*. github.com/huggingface/peft/issues/1569.
- [24] Agbavor, F., & Liang, H. (2022). *Predicting dementia from spontaneous speech using large language models*. PLOS Digital Health,

1(12):e0000168.

- [25] Zhu, Y., Obyat, A., Liang, X., Batsis, J. A., & Roth, R. M. (2021). *WavBERT: Exploiting Semantic and Non-semantic Speech using Wav2vec and BERT for Dementia Detection*. Interspeech 2021, 3790–3794. doi:10.21437/interspeech.2021-332.
- [26] (2025). *Statistical, multi-scale and attention-based layer pooling of Wav2Vec-2 speech embeddings for Parkinson's disease detection*. Computers in Biology and Medicine, ScienceDirect S0010482525017226.
- [27] (2023). *Wav2vec-based Detection and Severity Level Classification of Dysarthria from Speech*. arXiv:2309.14107.
- [28] Tao, F., Mirheidari, B., Pahar, M., et al. (2025). *Early Dementia Detection Using Multiple Spontaneous Speech Prompts: The PROCESS Challenge*. ICASSP 2025.
- [29] (2024). *Leveraging Multimodal Methods and Spontaneous Speech for Alzheimer's Disease Identification*. arXiv:2412.09928.
- [30] Zafar, A., Zhang, Y., Shahin, M., & Ahmed, B. (2025). *Multi-Class Dementia Detection Using Acoustic Features — ICASSP-2025 PROCESS Challenge*. ICASSP 2025.
- [31] (2024). *Structured Speaker-Deficiency Adaptation of Foundation Models for Dysarthric and Elderly Speech Recognition*. arXiv:2412.18832.
- [32] Vaswani, A., et al. (2017). *Attention Is All You Need*. NeurIPS 30.